

Pandas & Tabular Data Notes

Core Concepts: DataFrame vs. Series, Constructors, Inspection Attributes, Indexing (.loc & .iloc), Filtering, and In-Place Mutations

1. Introduction: What is Pandas For?

In our previous notes on Python OOPS and NumPy, we saw two extremes: standard Python objects can be heterogeneous (holding strings, numbers, and lists together) but suffer from severe memory overhead and slow loops; NumPy arrays solve this with blazing C-speed vectorization, but strictly enforce homogeneity—meaning every single cell in an ndarray must have the exact same data type (e.g., all float64 or all int64).

Real-world data is tabular and heterogeneous. In an Excel sheet, SQL database, or CSV file, each record has mixed types: a customer's Name (string), Age (integer), Account Balance (float), and Active Status (boolean). NumPy cannot manage this natively without coercing everything into generic string objects.

This is where Pandas comes in: it is a library built directly on top of NumPy that provides flexible, labeled, tabular data structures designed for real-world datasets. It preserves column-by-column data types while keeping fast C-level vectorized performance.

2. The Two Primary Classes: Series vs. DataFrame

Just like Bugatti or Student were custom classes in our OOPS notes, Pandas provides two core classes that you will instantiate and manipulate continuously:

- **pd.Series:** A 1-Dimensional labeled array capable of holding any data type. It consists of two coordinated components: an array of actual data values and an associated array of row labels called the Index. Think of a Series as an individual column in a spreadsheet.
- **pd.DataFrame:** A 2-Dimensional tabular data structure with labeled axes (rows and columns). Under the hood, a DataFrame is essentially an aligned dictionary of Series objects where every Series shares the exact same row index.

Key Mental Model

Relationship between Series & DataFrame:

Every column inside a DataFrame is an instance of a `pd.Series` object! If you extract a single column from a DataFrame, you receive a Series. If you stitch multiple Series sharing the same row index together side-by-side, you form a DataFrame.

3. Creating Objects: Constructors & `read_csv()`

Remember that in Python, calling `ClassName()` invokes the class constructor (`__init__`) to allocate memory and initialize data members. We can build Series and DataFrames manually using their constructors, or ingest them directly from external files:

```
import pandas as pd
import numpy as np

# --- 1. Creating a Series with pd.Series(data, index=...) ---
# Default automatic 0-indexed integer labels:
s_default = pd.Series([92, 85, 96, 78])

# Custom labeled index (like key-value lookup):
s_custom = pd.Series([9.5, 8.2, 7.8], index=["Mike", "Harvey", "Donna"], name="GPA")
```

```

print("Series with custom index:")
print(s_custom)
# Mike    9.5
# Harvey   8.2
# Donna    7.8
# Name: GPA, dtype: float64

# --- 2. Creating a DataFrame with pd.DataFrame(data, ...) ---
# The most common way: Dictionary of lists (keys = column names, values = column data)
data = {
    "Name": ["Mike", "Harvey", "Donna", "Rachel"],
    "Role": ["Associate", "Partner", "COO", "Paralegal"],
    "Hourly_Rate": [250, 600, 450, 180],
    "Active": [True, True, True, False]
}

df = pd.DataFrame(data)
print("\nDataFrame from constructor:")
print(df)

# --- 3. Ingesting Real-World Files via pd.read_csv() ---
# In practice, data is loaded directly from CSV files into DataFrame instances:
# df_csv = pd.read_csv("law_firm_staff.csv")
# Optional: Set a specific column as row labels directly on load:
# df_csv = pd.read_csv("law_firm_staff.csv", index_col="Employee_ID")

```

4. Inspecting Properties & Basic Metadata

Just as a Student object in our OOPS code contained data members like `self.Name` and `self.roll_no`, a DataFrame object contains built-in attributes (properties) and methods to quickly inspect its structure, dimensions, and types:

Member / Method	Type	Description & Output Example
.shape	Attribute (data member)	Tuple representing (rows, columns). E.g. (4, 4)
.ndim	Attribute (data member)	Total dimensions. Always 2 for DataFrame, 1 for Series
.dtypes	Attribute (data member)	Data type of each individual column (int64, float64, object)
.columns / .index	Attribute (data member)	Index objects holding column headers and row labels
.head(n) / .tail(n)	Method (function)	Returns top/bottom n rows (default 5) for quick preview
.info() / .describe()	Method (function)	Comprehensive summaries: memory, null counts, mean, min, max

```

# Inspecting our DataFrame instance:
print("df.shape:  ", df.shape)    # (4, 4) -> 4 rows, 4 columns
print("df.ndim:   ", df.ndim)     # 2
print("df.size:   ", df.size)     # 16 total elements
print("df.columns: ", list(df.columns))# ['Name', 'Role', 'Hourly_Rate', 'Active']
print("\ndf.dtypes:")
print(df.dtypes)
# Name      object (strings stored as object)
# Role      object
# Hourly_Rate  int64
# Active     bool
# dtype: object

# Quick method calls:
print("\nFirst 2 rows via df.head(2):")
print(df.head(2))

# Statistical summary of numerical columns:
print("\nSummary statistics via df.describe():")
print(df.describe())

```

5. Selecting Parts of a DataFrame

Selecting data in Pandas requires choosing between column extraction, label-based indexing (`.loc`), positional integer slicing (`.iloc`), and conditional boolean masks:

```

# Assume custom row indices for crystal-clear illustration:
df.index = ["emp_101", "emp_102", "emp_103", "emp_104"]

# =====
# A. COLUMN SELECTION (Bracket Syntax)
# =====
# Single bracket returns a 1D Series:
names_series = df["Name"]

# Double bracket returns a 2D DataFrame:
subset_df = df[["Name", "Hourly_Rate"]]

# =====
# B. LABEL-BASED INDEXING: .loc[row_label, col_label]
# Rule: Inclusive of both START and STOP boundaries!
# =====
# Select scalar value:
rate = df.loc["emp_101", "Hourly_Rate"]    # Returns 250

# Slicing rows from emp_101 to emp_103 and specific columns:
staff_slice = df.loc["emp_101":"emp_103", ["Name", "Role"]]

```

```

# =====
# C. POSITION-BASED INDEXING: .iloc[row_idx, col_idx]
# Rule: Python 0-indexed; STOP boundary is EXCLUDED!
# =====
# Select row 0, column 1 ('Associate'):
pos_val = df.iloc[0, 1]

# Slice first 2 rows (0, 1) and first 3 columns (0, 1, 2):
matrix_slice = df.iloc[0:2, 0:3]

# =====
# D. CONDITIONAL SELECTION (Boolean Masking)
# =====
# Step 1: Evaluating a comparison produces a Boolean Series
mask = df["Hourly_Rate"] >= 300
# Output:
# emp_101    False
# emp_102     True
# emp_103     True
# emp_104    False
# Name: Hourly_Rate, dtype: bool

# Step 2: Pass the boolean mask back into df[...] to filter rows
high_earners = df[mask]

# Multi-condition filtering (MUST use '&' for AND, '|' for OR, with parentheses):
partners_or_expensive = df[(df["Hourly_Rate"] >= 400) & (df["Active"] == True)]

```

Indexing Rule of Thumb

Critical Difference: .loc vs .iloc

- .loc is label-oriented: df.loc['a':'c'] includes 'c' (inclusive end).
- .iloc is zero-offset positional: df.iloc[0:2] grabs rows 0 and 1 only (exclusive end, exactly like standard Python slicing [0:2]).

6. Updating DataFrames: Mutations, New Columns & Drops

DataFrames are mutable objects. You can modify specific cells, compute whole columns in a single vectorized statement, or drop unwanted rows and columns:

```

# =====
# A. UPDATING EXISTING CELLS
# =====
# Update a single cell using .loc:
df.loc["emp_101", "Hourly_Rate"] = 280

# Conditional bulk update: Give inactive staff a 0 hourly rate
df.loc[df["Active"] == False, "Hourly_Rate"] = 0

```

```
# =====
# B. ADDING NEW COLUMNS
# =====
# Scalar assignment (broadcasts across every row automatically):
df["Currency"] = "USD"

# Vectorized arithmetic from existing columns (No for loops required!):
df["Overtime_Rate"] = df["Hourly_Rate"] * 1.5

# =====
# C. DROPPING ROWS & COLUMNS (.drop)
# =====
# Drop column (axis=1 or columns=[...]):
# By default, .drop() returns a brand-new copy without modifying the original:
df_without_curr = df.drop(columns=["Currency"])

# Permanent modification: Use inplace=True
df.drop(columns=["Currency"], inplace=True)

# Drop rows by index label (axis=0):
df.drop(index=["emp_104"], inplace=True)

print("Final Mutated DataFrame:")
print(df)
```

Deep Dive: .loc vs. .iloc and Slicing Semantics

Label-Based vs. Positional Indexing, Boundary Rules, Multi-Axis Slicing, and Integer Index Pitfalls

The most common source of bugs in Pandas comes from confusing label-based indexing (.loc) with integer position-based indexing (.iloc). While both follow the 2D coordinate pattern [row_indexer, column_indexer], their internal rules for slicing, boundary inclusion, and error handling work in fundamentally different ways.

1. Why Two Separate Indexers Exist

In standard Python lists or NumPy arrays, positions are always implicit 0-based integers (0, 1, 2, ...). In Pandas, however, a DataFrame has an explicit Index (which can consist of custom strings, timestamps, or non-sequential IDs like 1052, 2041, 3099).

If a user asks Pandas for `df[1]`, it creates immediate ambiguity: does 1 refer to the row at physical offset 1 (the 2nd row), or the row whose index label is the integer 1? To eliminate this confusion, Pandas splits access into two explicit indexers:

- `.loc[...]`: Strictly Label-based. It queries the explicit index labels and column names printed on your table.
- `.iloc[...]`: Strictly Integer position-based. It completely ignores labels and references zero-indexed physical memory offsets (0 to N - 1), exactly like standard Python list indexing.

2. The Crucial Difference in Slicing Boundaries

The fundamental difference between .loc and .iloc lies in how slice ranges (start:stop) are bounded:

- `.iloc[start:stop]` uses Python slicing conventions (Exclusive End):
 - Slicing `0:3` selects offsets 0, 1, and 2. Offset 3 is excluded.
 - Total elements returned = stop - start (`3 - 0 = 3`).
- `.loc[start:stop]` uses Dictionary/Inclusive conventions (Inclusive End):
 - Slicing `'A':'C'` selects row 'A', row 'B', and row 'C'. The stop label 'C' is included.
 - Why? Row labels are arbitrary strings or timestamps. If you slice dates from `'2026-01-01':'2026-01-31'`, you expect the 31st to be included. If the stop boundary were excluded, you would have to guess the label that immediately succeeds `'2026-01-31'`, which is impossible in non-continuous datasets.

3. Code Walkthrough: Side-by-Side Comparison

```
import pandas as pd

# DataFrame with custom non-sequential string labels:
data = {
    "Department": ["HR", "Engineering", "Marketing", "Finance"],
    "Salary": [65000, 120000, 85000, 95000]
}
df = pd.DataFrame(data, index=["emp_a", "emp_b", "emp_c", "emp_d"])

# -----
# A. Row Slicing: Inclusive vs. Exclusive
# -----
# .loc includes both "emp_a" AND "emp_c" (3 rows total)
slice_loc = df.loc["emp_a":"emp_c"]
```

```

# Returns: emp_a, emp_b, emp_c

# .iloc slices indices 0 up to (but not including) 2 (2 rows total)
slice_iloc = df.iloc[0:2]
# Returns: emp_a (offset 0), emp_b (offset 1)

# -----
# B. Slicing Both Rows and Columns Simultaneously
# -----
# Using .loc: labels for both axes (both endpoints inclusive)
sub_loc = df.loc["emp_b":"emp_d", "Department":"Salary"]

# Using .iloc: integer offsets for both axes (both endpoints exclusive)
# Rows: 1 up to 3 (rows 1, 2) | Cols: 0 up to 1 (col 0 only)
sub_iloc = df.iloc[1:3, 0:1]

# -----
# C. Stepping and Negative Indices
# -----
# .iloc natively supports negative indexing (like Python lists)
last_row = df.iloc[-1, :] # Extracts the final row (emp_d)
every_second = df.iloc[::2, :] # Skips every other row (0, 2)

# .loc DOES NOT support negative integers unless the literal label is -1!
# df.loc[-1, :] -> Raises KeyError!

```

4. The Special Edge Case: Numeric Non-Sequential Indices

Confusion peaks when a DataFrame's index contains numbers that are not standard consecutive 0, 1, 2 series (such as database IDs or sorted values):

```

# A DataFrame whose index labels are integers, but NOT 0, 1, 2...
df_num = pd.DataFrame({"Score": [88, 92, 79]}, index=[10, 20, 30])

# 1. df_num.loc[10:20]
# Looks for LABELS 10 through 20 -> Returns 2 rows (labels 10 and 20)

# 2. df_num.iloc[10:20]
# Raises IndexError! (Because offsets 10 through 20 do not exist in a 3-row table)

# 3. df_num.iloc[0:2]
# Looks for POSITIONS 0 and 1 -> Returns the first 2 rows (labels 10 and 20)

# 4. df_num.loc[0:2]
# Returns an EMPTY DataFrame! (Because no labels 0, 1, or 2 exist in the index)

```

- Reach for `.loc` when your logic depends on WHAT items are named (labels, dates, employee IDs).
- Reach for `.iloc` when your logic depends on WHERE items physically sit (e.g., 'first 10 rows and last 2 columns').

7. Comparative Architectural Summary

Dimension / Feature	NumPy ndarray	Pandas DataFrame
Data Types	Strictly Homogeneous (all elements same dtype)	Heterogeneous (columns have distinct dtypes)
Axis Labeling	Implicit zero-based numerical indexing only	Explicit index labels for rows & named columns
Primary Use-Case	Raw matrix math, linear algebra, tensors	Tabular analysis, CSV/SQL data, filtering & cleaning
Missing Data	Limited native handling (np.nan in float arrays)	Built-in first-class support (isna, fillna, dropna)